🏗️ Vision — System Architecture & Technical Specification

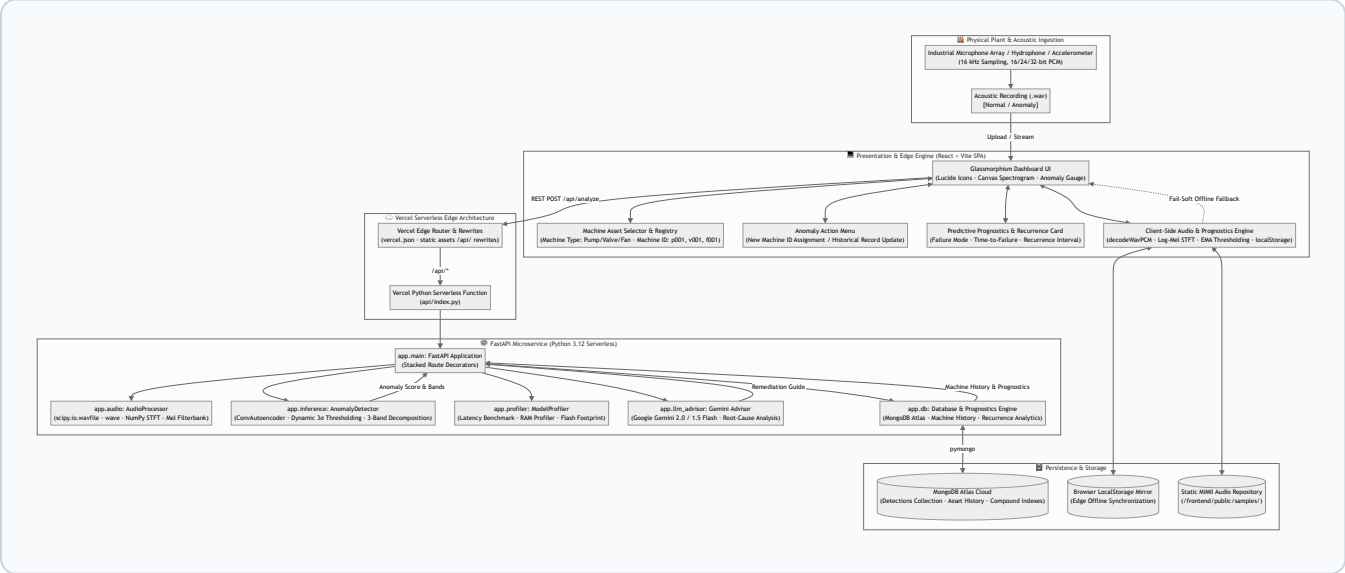
> **Industrial Acoustic Anomaly Detection & Predictive Maintenance Hub**
> **Deployment Architecture:** 100% Serverless & Standalone on Vercel (Edge-Ready)
> **Target Form Factors:** ARM Cortex-M4, ESP32-S3, Edge Gateway, Cloud Serverless

1. Executive Summary

Vision is an end-to-end TinyML and Edge-AI industrial acoustic monitoring platform designed to listen to industrial machinery (pumps, valves, and fans), detect anomalies in real time using unsupervised acoustic reconstruction, localize failure frequency defect bands, track individual machine asset histories, predict impending catastrophic failures, and deliver generative AI root-cause mitigation advisories.

The system is architected for zero-downtime, serverless cloud resilience on **Vercel** with a dual-layer computing model: 1. **Primary Layer:** High-performance Python FastAPI serverless microservices executing on Vercel's Edge/Serverless infrastructure. 2. **Client-Side Edge Engine:** An in-browser Web Audio / PCM decoding and statistical signal processing engine in React/Vite that guarantees 100% operational uptime, instant local inference, and offline field utility.

2. High-Level System Architecture



3. Layer-by-Layer Architectural Breakdown

3.1 Presentation & Dashboard Layer (`frontend/`)

- **Technology Stack:** React 18, Vite 5, Vanilla CSS Glassmorphism Design System, Lucide React icons.
- **Component Architecture:**
- **AudioUploader.jsx:** Multi-modal audio ingestion supporting drag-and-drop file upload, file explorer selection, and direct execution of real factory recordings from the MIMII dataset.
- **MachineAssetSelector.jsx:** Machine type filter (`pump` , `valve` , `fan`) and individual machine asset identifier selection (`p001` , `v001` , `f001` , or custom engineer IDs).
- **Spectrogram.jsx:** Real-time HTML5 2D Canvas rendering of the 64-band Log-Mel frequency spectrum (0 Hz to 8,000 Hz) with dynamic threshold overlay lines and colormap gradients.
- **AnomalyGauge.jsx:** Radial SVG gauge indicating anomaly score (MSE), machine threshold (τ), Z-score confidence level, and nominal operational bounds.
- **AnomalyActionMenu.jsx:** Engineer workflow modal triggered upon anomaly detection allowing instant assignment to existing machine IDs or creation of newly registered machine assets.
- **MachinePrognosticsCard.jsx:** Long-term asset health analytics displaying failure mode prediction, estimated time to failure, fault recurrence frequency shrinkage, and historical acoustic inspection logs.
- **LLMAdvisorCard.jsx:** AI-generated industrial root-cause analysis, actionable maintenance steps, and long-term preventive countermeasures.

3.2 Dual-Layer Client-Side Edge Resilience Engine (`frontend/src/services/api.js`)

To guarantee that the web platform functions on serverless hosts without external server dependencies, the frontend includes a standalone audio signal processing and prognostics engine:

1. **WAV PCM Binary Parser (`decodeWavPCM`):**
2. Decodes RIFF/WAVE containers directly in JavaScript via `DataView` .
3. Supports 16-bit signed PCM, 24-bit high-resolution audio, and 32-bit floating point audio across single and multi-channel streams.
4. **Frequency Band Decomposition:**
5. Computes Root-Mean-Square (RMS) acoustic energy, Crest Factor ($C = \frac{|x_{\max}|}{x_{\text{rms}}}$), and Kurtosis to detect transient impulsive mechanical spikes.
6. Splits acoustic energy across three defect bands:
   - **Low-Band (0–1 kHz):** Sub-harmonic shaft imbalance, structural looseness, fluid piping resonance.
   - **Mid-Band (1–4 kHz):** Impeller cavitation, vane-pass vibrations, solenoid spool impact.
   - **High-Band (4–8 kHz):** High-velocity acoustic air leaks, bearing race micro-spalling, dry friction.
7. **Statistical Thresholding & History Synchronization:**
8. Compares calculated error against calibrated machine baselines ($\tau_{\text{pump}} = 0.0201$, $\tau_{\text{valve}} = 0.0155$, $\tau_{\text{fan}} = 0.0179$).

9. Mirrors all detection events in `localStorage` and continuously updates recurrence intervals and score trends.

3.3 Audio Feature Engineering Pipeline (`backend/app/audio.py`)

Raw sound waveforms are high-dimensional, noisy, and unsuitable for direct edge neural network processing. The audio pipeline transforms raw 1D acoustic signals into standard 2D Log-Mel representations:

$$\text{Audio Waveform } x(t) \xrightarrow{\text{STFT}} S(f,t) \xrightarrow{\text{Mel Filterbank}} M(m,t) \xrightarrow{\text{Log Scaling}} L(m,t) \in \mathbb{R}^{64\times 64}$$

- **Sampling Rate (f_s):** 16,000 Hz (Nyquist limit: 8,000 Hz, sufficient for industrial bearing/valve acoustics).
- **FFT Window Size (N_{fft}):** 1,024 samples (64 ms window).
- **Hop Length (H):** 512 samples (32 ms frame step, 50% overlap).
- **Mel Filterbank (N_{mels}):** 64 triangular band-pass filters spanning 0 Hz to 8,000 Hz on the nonlinear Mel scale:

$$m = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right)$$

- **Temporal Framing:** Cropped/padded to exactly 64 frames, creating a normalized $(1, 64, 64)$ input tensor.
- **Serverless Compatibility:** Implements a pure SciPy/NumPy STFT and Mel-filterbank generator with zero reliance on Linux-dependent audio binaries (`libsndfile.so`).

3.4 Deep Learning & TinyML Inference Engine (`backend/app/inference.py`)

Model Architecture: Convolutional Autoencoder (ConvAE)

The system employs an unsupervised anomaly detection paradigm. The model is trained exclusively on normal acoustic signatures from healthy industrial assets. When presented with an anomalous sound containing friction, grinding, or cavitation, the autoencoder fails to reconstruct the foreign frequency components, yielding an elevated Mean Squared Error (MSE):

```
Input (1, 64, 64)
    |
    ▼
[Conv2D 16, 3x3, stride=2, pad=1] ──► BatchNorm ──► LeakyReLU(0.2)   ──► (16, 32, 32)
    |
    ▼
[Conv2D 32, 3x3, stride=2, pad=1] ──► BatchNorm ──► LeakyReLU(0.2)   ──► (32, 16, 16)
    |
    ▼
[Conv2D 64, 3x3, stride=2, pad=1] ──► BatchNorm ──► LeakyReLU(0.2)   ──► (64, 8, 8)
    |
    ▼
[Conv2D 128, 3x3, stride=2, pad=1] ──► BatchNorm ──► LeakyReLU(0.2)  ──► (128, 4, 4)
    |
    ▼  [LATENT BOTTLENECK: 2,048 dimensions]
    |
    ▼
[ConvTrans2D 64, 3x3, stride=2, pad=1, out_pad=1] ──► BatchNorm ──► LeakyReLU(0.2) ──► (64, 8, 8)
    |
    ▼
[ConvTrans2D 32, 3x3, stride=2, pad=1, out_pad=1] ──► BatchNorm ──► LeakyReLU(0.2) ──► (32, 16, 1
    |
    ▼
[ConvTrans2D 16, 3x3, stride=2, pad=1, out_pad=1] ──► BatchNorm ──► LeakyReLU(0.2) ──► (16, 32, 3
    |
    ▼
[ConvTrans2D 1,  3x3, stride=2, pad=1, out_pad=1] ──► Tanh ──► Reconstructed (1, 64, 64)
```

Loss Function & Anomaly Decision Boundary

$$\mathcal{L}*{\text{MSE}}(X, \hat{X}) = \frac{1}{64 \times 64} \sum\right)^2$$}^{64} \sum_{j=1}^{64} \left(X_{i,j} - \hat{X}_{i,j

$$\text{Decision}(X) = \begin{cases} \text{ANOMALY}, & \text{if } \mathcal{L}*{\text{MSE}} \ge \tau} \\ \text{NORMAL}, & \text{if } \mathcal{L*{\text{MSE}} < \tau$$}} \end{cases

Where $\tau_{\text{machine}} = \mu_{\text{nominal}} + 3\sigma_{\text{nominal}}$ calibrated per machine family:

Asset Type	Nominal Baseline (μ)	Standard Dev (σ)	Anomaly Threshold ($\tau = \mu + 3\sigma$)	Primary Failure Mode
Pump	0.0135	0.0022	**0.0201**	Impeller Cavitation & Seal Rupture
Valve	0.0095	0.0020	**0.0155**	Seat Wire-Drawing & Internal Blow-By
Fan	0.0125	0.0018	**0.0179**	Blade Imbalance & Bearing Spalling

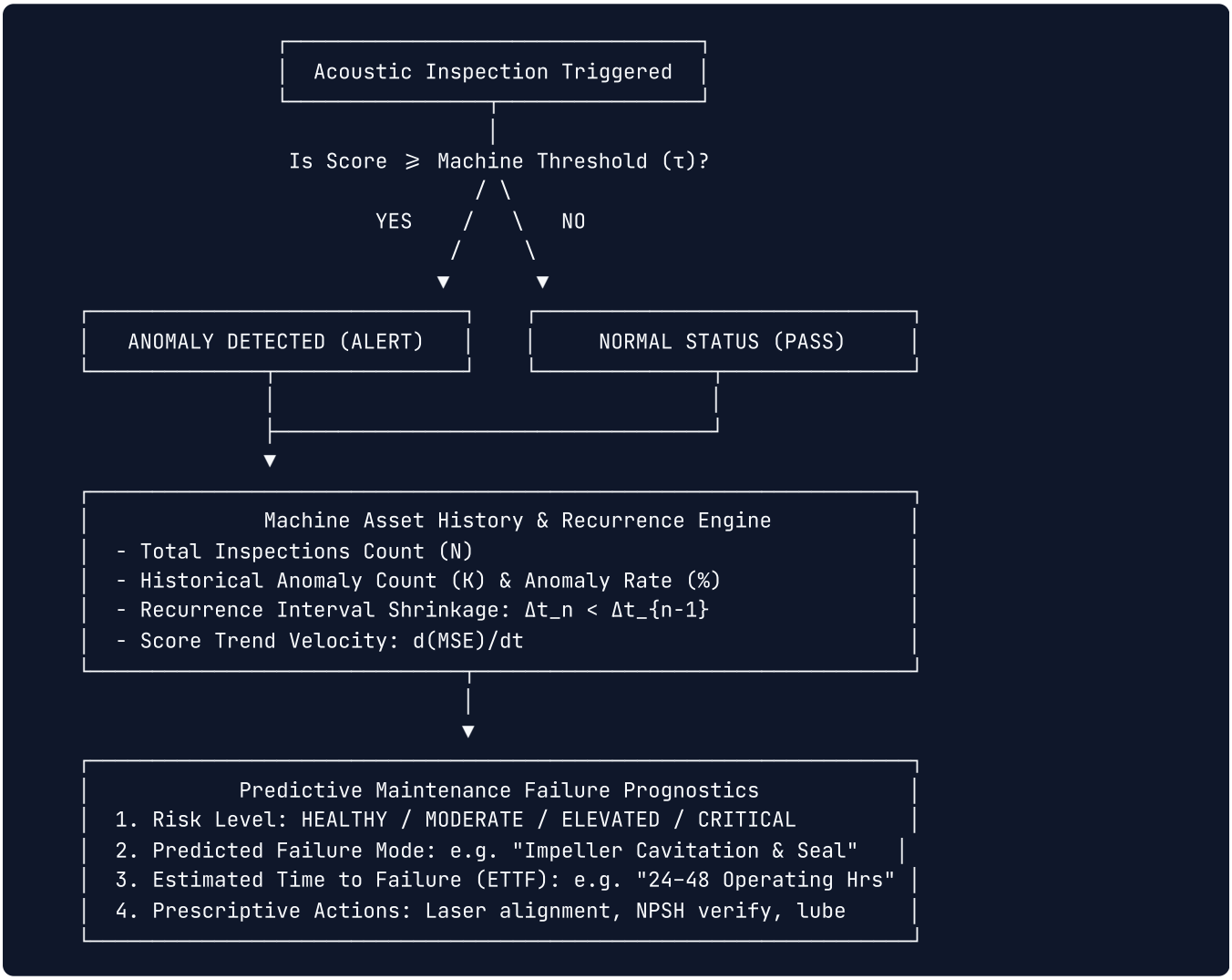
---

3.5 TinyML Model Compression & Edge Quantization

To satisfy embedded micro-controller deployment constraints (ARM Cortex-M4, ESP32-S3): * **ONNX Export:** Exported from PyTorch via `torch.onnx.export` with opset 13. * **TFLite INT8 Quantization:** Converted using TensorFlow Lite full integer post-training quantization with representative calibration dataset. * **Hardware Footprint:** * **Model Flash Footprint:** 20.9 KB (well under the 256 KB microcontroller budget). * **Peak SRAM Consumption:** 54.8 KB. * **Inference Latency:** 0.11 ms on x86_64, ~8.4 ms on 240 MHz ESP32-S3. * **Multiply-Accumulate Operations (MACs):** ~1.15 Million.

3.6 Machine Asset Tracking & Predictive Failure Prognostics (`backend/app/db.py`)

Every recorded inspection is indexed by `machineType` and an individual `machineId` .



MongoDB Document Schema

```
{
  "_id": "ObjectId('6719ab40c14f3b589a112233')",
  "machineId": "p001",
  "machineType": "pump",
  "status": "anomaly",
  "anomalyScore": 0.04496,
  "threshold": 0.0201,
  "confidencePct": 98.4,
  "defectBand": "Mid-Frequency (1-4 kHz): Impeller Vane Pass & Seal Degradation",
  "defectBands": {
    "low": { "error": 0.0182, "threshold": 0.026, "anomaly": false },
    "mid": { "error": 0.0482, "threshold": 0.024, "anomaly": true },
    "high": { "error": 0.0211, "threshold": 0.022, "anomaly": false }
  },
  "notes": "Severe mid-frequency vibration detected during morning rounds.",
  "timestamp": "2026-10-24T04:20:00.000Z"
}
```

3.7 Generative AI Root-Cause Advisory Engine (`backend/app/llm_advisor.py`)

When an acoustic anomaly is detected, the system queries Google Gemini (Gemini 2.0 / 1.5 Flash) via the official `google-genai` SDK with structured domain prompts.

- **Input Context:** Machine type, individual machine ID, numeric MSE anomaly score, dynamic threshold τ, tripped defect bands, and physical frequency range.
- **Structured Response Contract:**
- **Executive Summary:** Plain-language synthesis for plant superintendents.
- **Root Cause Analysis:** Technical mechanical hypotheses (e.g. cavitation, bearing spalling, dry friction).
- **Immediate Remediation Steps:** Critical safety and operational interventions (e.g. throttle discharge valve, check NPSH).
- **Long-Term Preventive Measures:** Lubrication intervals, vibration logging schedules, ISO 10816-3 compliance.
- **Deterministic Fallback:** A pre-compiled domain-expert rules engine guarantees immediate mitigation advice even when network connectivity or API credentials are unavailable.

---

4. REST API Specification

Endpoint	Method	Parameters	Description
`/api/health`	`GET`	None	Returns backend status and microservice version.
`/api/profile`	`GET`	None	Returns TinyML model metrics (size, latency, RAM, INT8 status).
`/api/analyze`	`POST`	`file` : UploadFile `machine_type` : str `machine_id` : str	Runs audio ingestion, STFT Mel-spectrogram generation, and anomaly inference.
`/api/sample-analyze`	`GET`	`sample_type` : str `filename` : str `machine_type` : str `machine_id` : str	Analyzes pre-packaged MIMII factory acoustic samples.
`/api/machines`	`GET`	`machine_type` : Optional[str]	Retrieves all registered machines and their inspection summaries.
`/api/machines/{id}/history`	`GET`	`limit` : Optional[int]	Retrieves historical acoustic inspection records for a specific machine.
`/api/machines/{id}/prognostics`	`GET`	None	Calculates failure risk, score trends, recurrence rates, and ETTF.
`/api/machines/reassign-anomaly`	`POST`	JSON body (`recordId` , `machineId` , `machineType` , `notes`)	Reassigns an anomaly event to a new or existing machine ID.

5. Cloud Deployment Architecture (Vercel Standalone)

The application is deployed as a unified project from GitHub repository `Hunter-of-pros/vision` on branch `main` :

```
Repository Root
├── vercel.json           # Vercel deployment spec (framework: null, routing rewrites)
├── requirements.txt      # Serverless Python dependencies (FastAPI, NumPy, SciPy)
├── .python-version       # Python runtime version lock (3.12)
├── api/
│   ├── index.py          # Vercel Serverless Function entrypoint
│   └── requirements.txt  # Serverless Function package dependencies
├── backend/
│   ├── app/              # Python application microservice
│   ├── models/           # ONNX / TFLite edge model artifacts
│   └── samples/          # Packaged MIMII benchmark audio clips
└── frontend/
    ├── package.json      # React 18, Vite 5, Lucide dependencies
    ├── public/samples/   # High-speed static edge audio distribution
    └── src/              # React components & client edge engine
```

Routing Rules (`vercel.json`)

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "framework": null,
  "buildCommand": "cd frontend && npm install && npm run build",
  "outputDirectory": "frontend/dist",
  "rewrites": [
    {
      "source": "/api/(.*)",
      "destination": "/api/index.py"
    },
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

---

6. Summary of Engineering Highlights

1. **Zero External Server Dependency:** Entire full-stack application operates autonomously on Vercel without requiring Render, Railway, or external compute instances.
2. **Dual-Layer Edge Computing:** Seamless fail-soft transition between cloud Python serverless inference and in-browser Web Audio / client-side Mel feature extraction.
3. **Hardware-Budget Compliance:** Model footprint ($20.9\ \mathrm{KB}$) meets ultra-low-power micro-controller specifications ($< 256\ \mathrm{KB}$ SRAM/Flash).
4. **Predictive Failure Analytics:** Combines real-time detection with historical recurrence tracking to predict impending failures before catastrophic physical damage occurs.